

MVP Versions — Build in Stages

ExamGuard AI — Implementation Guide

Version 0.1 — Phone Detector

BUILD THIS AFTER: Phase 5 (Object Detection & Classification)

What This Version Does

One camera. One job. Detect phones on desks during an exam.

That is it. No gaze tracking, no behavior analysis, no fancy dashboard. Just:

Camera → See phone → Print alert

This is your **FIRST** working product. It is simple, but it is **REAL** and **USEFUL**.

Why Start Here

Complex system with everything: 6 months to build, many things can fail

Phone detector only: 1-2 weeks to build, works immediately

A simple working product >> A complex unfinished product

A phone detector **ALONE** is useful because:

- Phones are the #1 cheating tool
- Easy to detect (phones have a distinct shape)
- Clear evidence (the phone is either there or not)
- Low false alarm rate (not much looks like a phone on a desk)

Tech Stack

Component	Technology	Purpose
Camera input	OpenCV	Read webcam or IP camera
Object detection	YOLOv8 (fine-tuned)	Detect phones on desks
Alert	Print to terminal	Show alert when phone detected

No database. No dashboard. No API. Just the basics.

Step-by-Step Build Guide

Step 1: Collect Training Data (2-3 days)

You need photos of phones on desks. About 500 images.

What to photograph:

- Phone face-up on desk (various angles)
- Phone face-down on desk
- Phone partially hidden under paper
- Phone in hand near desk

- Phone in pencil case (open)
- Empty desks (no phone) — for negative examples!

Vary the conditions:

- Different phone brands/sizes/colors
- Different desk colors and materials
- Different lighting (bright, dim, natural, artificial)
- Different camera angles (overhead, side, corner)

How to collect:

1. Take photos yourself: Put phone on desk, take photo from camera position
2. Ask friends: Different phones, different settings
3. Online: Search "phone on desk" on Google Images (supplement only)

Quick collection method:

```
import cv2
import os
import time
```

```
# Use webcam to collect training images
cap = cv2.VideoCapture(0)
output_dir = "training_data/phone_on_desk"
os.makedirs(output_dir, exist_ok=True)
```

```
count = 0
print("Press 's' to save frame, 'q' to quit")
print("Place phone on desk in different positions and press 's'")
```

```
while True:
    ret, frame = cap.read()
    cv2.imshow("Collector", frame)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('s'):
        count += 1
        filename = f"{output_dir}/phone_{count:04d}.jpg"
        cv2.imwrite(filename, frame)
        print(f"Saved: {filename}")
    elif key == ord('q'):
        break
```

```
cap.release()
print(f"Total images saved: {count}")
```

Step 2: Label the Data (1-2 days)

Use a labeling tool to draw boxes around phones in each image.

Tool: LabelImg (free, simple)

```
pip install labelimg
labelimg
```

Or: Roboflow (web-based, free for small datasets)

Go to roboflow.com, upload images, label online

For each image:

1. Open image
2. Draw a box around the phone
3. Label it "phone"
4. Save
5. Repeat 500 times (yes, it is tedious but essential!)

Step 3: Fine-Tune YOLOv8 (1 day)

```
from ultralytics import YOLO
```

```
# Start with pre-trained YOLOv8 nano (already knows many objects)
```

```
model = YOLO("yolov8n.pt")
```

```
# Fine-tune on your phone dataset
```

```
# Your dataset should be in YOLO format:
```

```
# dataset/
```

```
#   train/
```

```
#     images/  (80% of your photos)
```

```
#     labels/  (matching label files)
```

```
#   val/
```

```
#     images/  (20% of your photos)
```

```
#     labels/  (matching label files)
```

```
results = model.train(  
    data="dataset/data.yaml", # Path to dataset config  
    epochs=50,               # Train for 50 rounds  
    imgsz=640,               # Image size  
    batch=16,                # Batch size  
    name="phone_detector"    # Name for this training run  
)
```

```
# data.yaml should contain:
```

```
# train: dataset/train/images
```

```
# val: dataset/val/images
```

```
# nc: 1      # number of classes
```

```
# names: ['phone'] # class names
```

Step 4: Test the Model (1 hour)

```
from ultralytics import YOLO
```

```
import cv2
```

```
# Load your fine-tuned model
```

```
model = YOLO("runs/detect/phone_detector/weights/best.pt")
```

```
# Test on some images
```

```
results = model("test_images/", save=True)
```

```
# Check results
```

```
for r in results:
```

```

for box in r.bboxes:
    print(f"Detected: {model.names[int(box.cls)]}, "
          f"Confidence: {float(box.conf):.2f}")

```

Step 5: Build the Live Detector (1 day)

```

"""

```

ExamGuard v0.1 — Phone Detector

Run this to monitor a camera for phones on desks.

```

"""

```

```

import cv2
from ultralytics import YOLO
from datetime import datetime

# Load model
model = YOLO("runs/detect/phone_detector/weights/best.pt")

# Connect to camera (0 = webcam, or use RTSP URL)
cap = cv2.VideoCapture(0)

# Settings
CONFIDENCE_THRESHOLD = 0.7 # Only alert if > 70% confident
ALERT_COOLDOWN = 10       # Seconds between repeated alerts

last_alert_time = 0

print("=" * 50)
print("ExamGuard v0.1 — Phone Detector")
print("=" * 50)
print("Monitoring started. Press 'q' to quit.")
print()

while True:
    ret, frame = cap.read()
    if not ret:
        print("Camera error!")
        break

    # Run detection
    results = model(frame, verbose=False)

    # Check for phones
    phone_detected = False
    for box in results[0].bboxes:
        class_name = model.names[int(box.cls)]
        confidence = float(box.conf)

        if class_name == "phone" and confidence > CONFIDENCE_THRESHOLD:
            phone_detected = True

    # Draw box on frame

```

```

x1, y1, x2, y2 = map(int, box.xyxy[0])
cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 3)
cv2.putText(frame, f"PHONE {confidence:.0%}",
            (x1, y1-10), cv2.FONT_HERSHEY_SIMPLEX,
            0.9, (0, 0, 255), 2)

# Alert logic
current_time = datetime.now().timestamp()
if phone_detected and (current_time - last_alert_time) > ALERT_COOLDOWN:
    timestamp = datetime.now().strftime("%H:%M:%S")
    print(f"[{timestamp}] *** ALERT: Phone detected! "
          f"Confidence: {confidence:.0%} ***")

# Save evidence screenshot
evidence_name = f'evidence/phone_{datetime.now().strftime("%H%M%S")}.jpg'
cv2.imwrite(evidence_name, frame)
print(f"    Evidence saved: {evidence_name}")

last_alert_time = current_time

# Show status on frame
status = "PHONE DETECTED!" if phone_detected else "Monitoring..."
color = (0, 0, 255) if phone_detected else (0, 255, 0)
cv2.putText(frame, status, (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)

# Display
cv2.imshow("ExamGuard v0.1 - Phone Detector", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
print("\nMonitoring ended.")

```

Success Criteria

Metric	Target
Detect phone on desk	> 90% of the time
False alarm rate	< 5% (non-phone objects flagged)
Detection time	< 2 seconds after phone appears
Works with webcam	Yes
Saves evidence screenshot	Yes

What You Will Learn Building This

- 1. The full ML pipeline: data collection → labeling → training → deployment
- 2. How to fine-tune a pre-trained model on custom data
- 3. How to run AI on live camera feed
- 4. How to handle confidence thresholds

- 5. How to save evidence
- 6. The foundation for EVERYTHING that comes after

Limitations (What v0.1 Cannot Do)

Cannot detect: Gaze direction, body posture, passing notes,
hidden earpieces, coded signals, whispering

Cannot do: Multi-camera, dashboard, database, real-time alerts

These will be added in later versions!

After You Build This

Congratulations! You have a working AI product. It is simple but REAL.

Show it to people. Get feedback. Then move on to v0.2 (Gaze Tracker).

Each version adds a new capability, building on what you already have.

Version 0.2 — Gaze Tracker

BUILD THIS AFTER: Phase 5 (Object Detection & Classification)

What This Version Does

Single camera. Tracks where each student is looking. Alerts if a student stares at a neighbor's paper for too long.

Camera → Detect faces → Track eye direction → Measure duration → Alert if too long

This adds BEHAVIORAL detection to ExamGuard. v0.1 detected OBJECTS (phones). v0.2 detects BEHAVIOR (looking at neighbors).

Why Gaze Tracking

Most common cheating method: Looking at a neighbor's paper

How an invigilator catches it: Sees a student staring sideways for too long

ExamGuard v0.2 does the same thing, but never blinks or gets distracted

A quick glance (0.5 seconds) → normal, everyone does it.

A sustained stare (10+ seconds) at someone's paper → suspicious.

Repeated stares (5 times in 10 minutes) → very suspicious.

Tech Stack

Component	Technology	Purpose
Camera input	OpenCV	Read webcam or IP camera
Face detection	MediaPipe Face Mesh	Find face and 468 face landmarks
Eye detection	MediaPipe (built-in)	Locate eyes precisely
Gaze calculation	Custom math	Calculate where eyes point
Time tracking	Python	Measure gaze duration
Alert	Print + save evidence	Show alert when threshold exceeded

How Gaze Tracking Works

The Pipeline

Frame from camera

↓

MediaPipe Face Mesh → finds 468 points on face

↓

Extract eye landmarks (6 points per eye)

↓

Calculate iris position relative to eye corners

↓

Determine gaze direction: LEFT, RIGHT, CENTER, UP, DOWN

↓

Track duration: "Looking LEFT for 8.3 seconds..."

↓

If LEFT > 10 seconds → ALERT

The Math (Simplified)

Eye landmarks:

Left corner (P1) ←— eye —→ Right corner (P2)
(iris center)

Ratio = distance(iris, left_corner) / distance(left_corner, right_corner)

If ratio < 0.35 → Looking LEFT (eyes shifted to the left)

If ratio > 0.65 → Looking RIGHT (eyes shifted to the right)

If 0.35-0.65 → Looking CENTER (straight ahead)

Step-by-Step Build Guide

Step 1: Install MediaPipe

pip install mediapipe opencv-python numpy

Step 2: Detect Face and Eye Landmarks

```
import cv2
import mediapipe as mp
import numpy as np

# Initialize MediaPipe Face Mesh
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    max_num_faces=5,      # Track up to 5 faces
    refine_landmarks=True, # Include iris landmarks
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Convert to RGB (MediaPipe needs RGB)
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Detect face landmarks
    results = face_mesh.process(rgb_frame)

    if results.multi_face_landmarks:
        for face_landmarks in results.multi_face_landmarks:
            # There are 468 landmarks on the face
            # Draw them (for debugging)
            h, w = frame.shape[:2]
            for lm in face_landmarks.landmark:
```



```

        x, y = int(lm.x * w), int(lm.y * h)
        cv2.circle(frame, (x, y), 1, (0, 255, 0), -1)

    cv2.imshow("Face Mesh", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()

```

Step 3: Calculate Gaze Direction

```

# Key landmark indices for eyes:
# Left eye: [33, 160, 158, 133, 153, 144] (corners and edges)
# Right eye: [362, 385, 387, 263, 373, 380]
# Left iris: [468, 469, 470, 471, 472]
# Right iris: [473, 474, 475, 476, 477]

LEFT_EYE = [33, 160, 158, 133, 153, 144]
RIGHT_EYE = [362, 385, 387, 263, 373, 380]
LEFT_IRIS = [468, 469, 470, 471, 472]
RIGHT_IRIS = [473, 474, 475, 476, 477]

def get_gaze_direction(landmarks, frame_shape):
    """Calculate where the person is looking."""
    h, w = frame_shape[:2]

    # Get iris center (average of iris landmarks)
    left_iris_x = np.mean([landmarks[i].x for i in LEFT_IRIS]) * w
    right_iris_x = np.mean([landmarks[i].x for i in RIGHT_IRIS]) * w

    # Get eye corner positions
    left_eye_left = landmarks[33].x * w # Left corner of left eye
    left_eye_right = landmarks[133].x * w # Right corner of left eye
    right_eye_left = landmarks[362].x * w
    right_eye_right = landmarks[263].x * w

    # Calculate ratio for left eye
    left_eye_width = left_eye_right - left_eye_left
    if left_eye_width == 0:
        return "UNKNOWN"
    left_ratio = (left_iris_x - left_eye_left) / left_eye_width

    # Calculate ratio for right eye
    right_eye_width = right_eye_right - right_eye_left
    if right_eye_width == 0:
        return "UNKNOWN"
    right_ratio = (right_iris_x - right_eye_left) / right_eye_width

    # Average both eyes
    avg_ratio = (left_ratio + right_ratio) / 2

    # Determine direction

```

```

if avg_ratio < 0.35:
    return "LEFT"
elif avg_ratio > 0.65:
    return "RIGHT"
else:
    return "CENTER"

```

Step 4: Track Duration and Alert

```

import time
from collections import defaultdict

# Track gaze for each detected face
gaze_tracker = defaultdict(lambda: {
    'direction': 'CENTER',
    'start_time': time.time(),
    'total_left': 0,
    'total_right': 0,
    'alert_count': 0
})

GAZE_ALERT_THRESHOLD = 10 # Seconds of sustained side-gaze

def update_gaze(face_id, direction):
    """Track how long someone looks in each direction."""
    tracker = gaze_tracker[face_id]
    current_time = time.time()

    if direction != tracker['direction']:
        # Direction changed — calculate duration of previous gaze
        duration = current_time - tracker['start_time']

        if tracker['direction'] == 'LEFT':
            tracker['total_left'] += duration
        elif tracker['direction'] == 'RIGHT':
            tracker['total_right'] += duration

        # Reset timer for new direction
        tracker['direction'] = direction
        tracker['start_time'] = current_time
    else:
        # Same direction — check if threshold exceeded
        duration = current_time - tracker['start_time']

        if direction in ['LEFT', 'RIGHT'] and duration > GAZE_ALERT_THRESHOLD:
            return True, duration # ALERT!

    return False, 0

```

Step 5: Complete v0.2 System

```

"""
ExamGuard v0.2 — Gaze Tracker

```

Monitors where students look and alerts on sustained sideways gaze.

"""

```
import cv2
import mediapipe as mp
import numpy as np
import time
from datetime import datetime
import os

# Setup
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(
    max_num_faces=5,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

LEFT_IRIS = [468, 469, 470, 471, 472]
RIGHT_IRIS = [473, 474, 475, 476, 477]

cap = cv2.VideoCapture(0)
os.makedirs("evidence", exist_ok=True)

# Per-face tracking
face_gaze_start = {} # face_index → time when side-gaze started
ALERT_THRESHOLD = 10 # seconds

print("=" * 50)
print("ExamGuard v0.2 — Gaze Tracker")
print("=" * 50)
print(f"Alert threshold: {ALERT_THRESHOLD} seconds of sustained side-gaze")
print("Monitoring started. Press 'q' to quit.\n")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = face_mesh.process(rgb)
    h, w = frame.shape[:2]

    if results.multi_face_landmarks:
        for face_idx, face_lm in enumerate(results.multi_face_landmarks):
            lm = face_lm.landmark

            # Calculate gaze direction
            left_iris_x = np.mean([lm[i].x for i in LEFT_IRIS]) * w
            left_eye_left = lm[33].x * w
```

```

left_eye_right = lm[133].x * w
right_iris_x = np.mean([lm[i].x for i in RIGHT_IRIS]) * w
right_eye_left = lm[362].x * w
right_eye_right = lm[263].x * w

left_width = left_eye_right - left_eye_left
right_width = right_eye_right - right_eye_left

if left_width > 0 and right_width > 0:
    left_ratio = (left_iris_x - left_eye_left) / left_width
    right_ratio = (right_iris_x - right_eye_left) / right_width
    avg_ratio = (left_ratio + right_ratio) / 2

    if avg_ratio < 0.35:
        direction = "LEFT"
    elif avg_ratio > 0.65:
        direction = "RIGHT"
    else:
        direction = "CENTER"

# Track duration
looking_sideways = direction in ["LEFT", "RIGHT"]

if looking_sideways:
    if face_idx not in face_gaze_start:
        face_gaze_start[face_idx] = time.time()

    duration = time.time() - face_gaze_start[face_idx]

# Draw info
nose = lm[1]
cx, cy = int(nose.x * w), int(nose.y * h)

color = (0, 255, 255) if duration < ALERT_THRESHOLD else (0, 0, 255)
cv2.putText(frame, f"Looking {direction}: {duration:.1f}s",
            (cx - 80, cy - 40), cv2.FONT_HERSHEY_SIMPLEX,
            0.6, color, 2)

# ALERT if threshold exceeded
if duration > ALERT_THRESHOLD:
    timestamp = datetime.now().strftime("%H:%M:%S")
    print(f"[{timestamp}] *** ALERT: Face {face_idx} looking "
          f"{direction} for {duration:.1f}s ***")

evidence_name = f"evidence/gaze_{datetime.now().strftime('%H%M%S')}.jpg"
cv2.imwrite(evidence_name, frame)

# Draw red box around face
face_x = [int(lm[i].x * w) for i in range(468)]
face_y = [int(lm[i].y * h) for i in range(468)]
cv2.rectangle(frame,

```

```

        (min(face_x)-10, min(face_y)-10),
        (max(face_x)+10, max(face_y)+10),
        (0, 0, 255), 3)
    else:
        # Looking center — reset timer
        if face_idx in face_gaze_start:
            del face_gaze_start[face_idx]

        nose = lm[1]
        cx, cy = int(nose.x * w), int(nose.y * h)
        cv2.putText(frame, "Looking: CENTER",
                    (cx - 60, cy - 40), cv2.FONT_HERSHEY_SIMPLEX,
                    0.6, (0, 255, 0), 2)

# Status bar
cv2.putText(frame, "ExamGuard v0.2 - Gaze Tracker", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

cv2.imshow("ExamGuard v0.2", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Success Criteria

Metric	Target
Detect gaze direction	> 85% accuracy
Sustained side-gaze alert	Triggers after 10 seconds
False alarm rate	< 10%
Works with multiple faces	Up to 5 faces simultaneously
Saves evidence screenshot	Yes

Known Limitations

- Accuracy drops with glasses (reflections confuse iris detection)
- Does not work well if face is too far from camera
- Cannot tell the difference between looking at neighbor vs looking at wall
- Head rotation can confuse gaze vs head direction
- These will be improved in later versions with more sophisticated models

What You Will Learn

- 1. MediaPipe face mesh and landmark detection
- 2. Eye and iris tracking with computer vision
- 3. Geometric calculations for gaze direction
- 4. Time-based event tracking (duration monitoring)
- 5. Building on top of existing CV libraries

Next: Combine v0.1 + v0.2 into v0.3

After building this, you have TWO detection capabilities:

- Phone detection (v0.1)
- Gaze tracking (v0.2)

Version 0.3 will COMBINE these with body posture and anomaly detection into a unified behavior monitor.

Version 0.3 — Behavior Monitor

BUILD THIS AFTER: Phase 6 (Advanced ML)

What This Version Does

Single camera. Combines EVERYTHING from v0.1 and v0.2 plus body posture analysis and anomaly detection. For the first time, the system understands BEHAVIOR, not just objects or gaze.

Camera → YOLO (objects) + Gaze (eyes) + Pose (body) + Autoencoder (anomaly)

↓

Confidence scoring: Low / Medium / High

↓

Smart alert with evidence

This is the first version that feels like a REAL exam monitoring system.

What Makes v0.3 Different

v0.1: "I see a phone" → One signal

v0.2: "Student looking sideways" → One signal

v0.3: "Student looking sideways AND → Multiple signals COMBINED

leaning toward neighbor AND

hands not writing AND

this behavior is unusual for

this student"

→ CONFIDENCE: HIGH

The power is in COMBINING multiple signals. Each signal alone might be innocent. Together they tell a story.

Tech Stack

Component	Technology	Purpose
Camera input	OpenCV	Read camera feed
Object detection	YOLOv8 (fine-tuned)	Detect phones, notes, objects
Gaze tracking	MediaPipe Face Mesh	Where are eyes looking
Body pose	MediaPipe Pose	Body posture and movement
Anomaly detection	Autoencoder	Unusual behavior detection
Scoring engine	Custom Python	Combine signals into score
Alert system	Print + evidence save	Alert on high confidence

The Confidence Scoring System

How It Works

Each detection method gives a score. Scores are combined into an overall confidence level.

```
class ConfidenceScorer:
```

```
    """
```

```
    Combine multiple signals into a single confidence score.
```

```
    """
```

```
    def __init__(self):
```

```

# Weights: how much each signal matters
self.weights = {
    'phone_detected': 0.40,    # Strong evidence
    'gaze_at_neighbor': 0.25,  # Medium evidence
    'body_leaning': 0.15,     # Supporting evidence
    'hands_not_writing': 0.10, # Minor signal
    'anomaly_score': 0.10,    # Background check
}

# Thresholds for alert levels
self.thresholds = {
    'low': 0.3,
    'medium': 0.5,
    'high': 0.7,
}

def calculate(self, signals):
    """
    signals = {
        'phone_detected': 0.0 or confidence,
        'gaze_at_neighbor': 0.0 to 1.0 (based on duration),
        'body_leaning': 0.0 to 1.0 (based on angle),
        'hands_not_writing': 0.0 or 1.0,
        'anomaly_score': 0.0 to 1.0
    }
    """
    total = 0
    for signal, value in signals.items():
        weight = self.weights.get(signal, 0)
        total += weight * value

    # Determine alert level
    if total >= self.thresholds['high']:
        level = "HIGH"
    elif total >= self.thresholds['medium']:
        level = "MEDIUM"
    elif total >= self.thresholds['low']:
        level = "LOW"
    else:
        level = "NONE"

    return total, level

```

Examples

Scenario 1: Student stretching

phone_detected: 0.0

gaze_at_neighbor: 0.0

body_leaning: 0.3 (leaning back, not toward neighbor)

hands_not_writing: 1.0 (hands up stretching)

anomaly_score: 0.1

TOTAL: $0.0 + 0.0 + 0.045 + 0.1 + 0.01 = 0.155$

LEVEL: NONE ← Correct! Just stretching.

Scenario 2: Quick glance at neighbor

phone_detected: 0.0

gaze_at_neighbor: 0.3 (brief, < 3 seconds)

body_leaning: 0.0

hands_not_writing: 0.0 (still writing)

anomaly_score: 0.1

TOTAL: $0.0 + 0.075 + 0.0 + 0.0 + 0.01 = 0.085$

LEVEL: NONE ← Correct! Just a quick glance.

Scenario 3: Sustained gaze + leaning

phone_detected: 0.0

gaze_at_neighbor: 0.8 (8+ seconds)

body_leaning: 0.7 (leaning toward neighbor)

hands_not_writing: 1.0

anomaly_score: 0.5

TOTAL: $0.0 + 0.2 + 0.105 + 0.1 + 0.05 = 0.455$

LEVEL: MEDIUM ← Suspicious, worth watching

Scenario 4: Phone + gaze + leaning (everything)

phone_detected: 0.9

gaze_at_neighbor: 0.8

body_leaning: 0.6

hands_not_writing: 1.0

anomaly_score: 0.7

TOTAL: $0.36 + 0.2 + 0.09 + 0.1 + 0.07 = 0.82$

LEVEL: HIGH ← Very likely cheating!

Step-by-Step Build Guide

Step 1: Body Pose Detection with MediaPipe

```
import cv2
```

```
import mediapipe as mp
```

```
import numpy as np
```

```
mp_pose = mp.solutions.pose
```

```
pose = mp_pose.Pose(
```

```
    min_detection_confidence=0.5,
```

```
    min_tracking_confidence=0.5
```

```
)
```

```
def analyze_body_pose(frame):
```

```
    """Detect body posture and calculate lean angle."""
```

```
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
    results = pose.process(rgb)
```

```
    if not results.pose_landmarks:
```

```
        return 0, "UNKNOWN"
```

```
    lm = results.pose_landmarks.landmark
```

```

h, w = frame.shape[:2]

# Key points
left_shoulder = np.array([lm[11].x * w, lm[11].y * h])
right_shoulder = np.array([lm[12].x * w, lm[12].y * h])
nose = np.array([lm[0].x * w, lm[0].y * h])

# Shoulder midpoint
shoulder_mid = (left_shoulder + right_shoulder) / 2

# Lean angle: how far nose is from shoulder midpoint (horizontally)
lean_offset = nose[0] - shoulder_mid[0]

# Normalize: positive = leaning right, negative = leaning left
shoulder_width = np.linalg.norm(left_shoulder - right_shoulder)
if shoulder_width == 0:
    return 0, "UNKNOWN"

lean_ratio = lean_offset / shoulder_width

# Determine lean direction
if lean_ratio < -0.3:
    direction = "LEANING_LEFT"
elif lean_ratio > 0.3:
    direction = "LEANING_RIGHT"
else:
    direction = "UPRIGHT"

lean_score = min(abs(lean_ratio) / 0.5, 1.0) # 0 to 1

return lean_score, direction

```

Step 2: Hand Activity Detection

```

def analyze_hand_activity(pose_landmarks, frame_shape):
    """Check if hands are in writing position."""
    if not pose_landmarks:
        return True # Assume writing if cannot detect

    lm = pose_landmarks.landmark
    h, w = frame_shape[:2]

    # Wrist positions
    left_wrist = np.array([lm[15].x * w, lm[15].y * h])
    right_wrist = np.array([lm[16].x * w, lm[16].y * h])

    # Hip position (desk level reference)
    left_hip = np.array([lm[23].x * w, lm[23].y * h])
    right_hip = np.array([lm[24].x * w, lm[24].y * h])
    hip_level = (left_hip[1] + right_hip[1]) / 2

    # If wrists are near hip level → writing position

```

```

# If wrists are higher → not writing (reaching, gesturing, etc.)
left_writing = left_wrist[1] > hip_level * 0.7
right_writing = right_wrist[1] > hip_level * 0.7

is_writing = left_writing or right_writing
return is_writing

```

Step 3: Autoencoder for Anomaly Detection

```

import torch
import torch.nn as nn

class BehaviorAutoencoder(nn.Module):
    """Learns normal behavior, flags unusual patterns."""

    def __init__(self, input_dim=20):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 8), # Compress to just 8 numbers
        )
        self.decoder = nn.Sequential(
            nn.Linear(8, 32),
            nn.ReLU(),
            nn.Linear(32, 64),
            nn.ReLU(),
            nn.Linear(64, input_dim),
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

def extract_behavior_features(gaze_dir, lean_score, lean_dir, is_writing,
                             phone_detected, phone_conf):
    """Convert all signals into a feature vector for the autoencoder."""
    features = [
        1.0 if gaze_dir == "LEFT" else 0.0,
        1.0 if gaze_dir == "RIGHT" else 0.0,
        1.0 if gaze_dir == "CENTER" else 0.0,
        lean_score,
        1.0 if lean_dir == "LEANING_LEFT" else 0.0,
        1.0 if lean_dir == "LEANING_RIGHT" else 0.0,
        1.0 if lean_dir == "UPRIGHT" else 0.0,
        1.0 if is_writing else 0.0,
        phone_conf if phone_detected else 0.0,
        # Add more features as needed to reach input_dim
    ]

```

```

# Pad to input_dim
while len(features) < 20:
    features.append(0.0)

return torch.tensor(features, dtype=torch.float32)

def get_anomaly_score(autoencoder, features):
    """Higher score = more unusual behavior."""
    autoencoder.eval()
    with torch.no_grad():
        reconstructed = autoencoder(features.unsqueeze(0))
        error = torch.mean((features - reconstructed.squeeze()) ** 2).item()
    return min(error / 0.5, 1.0) # Normalize to 0-1

```

Step 4: Complete v0.3 System

```

"""
ExamGuard v0.3 — Behavior Monitor
Combines phone detection + gaze tracking + body pose + anomaly detection.
Multi-signal confidence scoring.
"""

import cv2
import mediapipe as mp
import numpy as np
from ultralytics import YOLO
from datetime import datetime
import time
import os

# Initialize all models
yolo_model = YOLO("phone_detector_model.pt")
mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(max_num_faces=5, refine_landmarks=True)
mp_pose = mp.solutions.pose
pose_detector = mp_pose.Pose()

# Initialize scorer
scorer = ConfidenceScorer() # From above

# Camera
cap = cv2.VideoCapture(0)
os.makedirs("evidence", exist_ok=True)

# Gaze tracking state
gaze_start_times = {}

print("=" * 50)
print("ExamGuard v0.3 — Behavior Monitor")
print("=" * 50)
print("Multi-signal monitoring active.\n")

```

```

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # — Signal 1: Phone Detection (YOLO) —
    yolo_results = yolo_model(frame, verbose=False)
    phone_detected = False
    phone_conf = 0.0
    for box in yolo_results[0].boxes:
        if yolo_model.names[int(box.cls)] == "phone" and float(box.conf) > 0.6:
            phone_detected = True
            phone_conf = float(box.conf)
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 2)

    # — Signal 2: Gaze Direction (MediaPipe Face Mesh) —
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    face_results = face_mesh.process(rgb)
    gaze_score = 0.0
    gaze_dir = "UNKNOWN"

    if face_results.multi_face_landmarks:
        for idx, face_lm in enumerate(face_results.multi_face_landmarks):
            gaze_dir = calculate_gaze(face_lm.landmark, frame.shape)

            if gaze_dir in ["LEFT", "RIGHT"]:
                if idx not in gaze_start_times:
                    gaze_start_times[idx] = time.time()
                duration = time.time() - gaze_start_times[idx]
                gaze_score = min(duration / 15.0, 1.0) # Max at 15 seconds
            else:
                gaze_start_times.pop(idx, None)
                gaze_score = 0.0

    # — Signal 3: Body Pose (MediaPipe Pose) —
    pose_results = pose_detector.process(rgb)
    lean_score = 0.0
    is_writing = True

    if pose_results.pose_landmarks:
        lean_score, lean_dir = analyze_body_pose(frame)
        is_writing = analyze_hand_activity(pose_results.pose_landmarks, frame.shape)

    # — Signal 4: Anomaly Score (Autoencoder) —
    # (Use 0.0 if autoencoder not trained yet)
    anomaly_score = 0.0

    # — Combine All Signals —
    signals = {
        'phone_detected': phone_conf if phone_detected else 0.0,

```

```

        'gaze_at_neighbor': gaze_score,
        'body_leaning': lean_score,
        'hands_not_writing': 0.0 if is_writing else 1.0,
        'anomaly_score': anomaly_score,
    }

    total_score, alert_level = scorer.calculate(signals)

    # — Display —
    colors = {"NONE": (0, 255, 0), "LOW": (0, 255, 255),
             "MEDIUM": (0, 165, 255), "HIGH": (0, 0, 255)}
    color = colors.get(alert_level, (255, 255, 255))

    # Status bar
    cv2.putText(frame, f"ExamGuard v0.3 | Score: {total_score:.2f} | {alert_level}",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

    # Signal details
    y_pos = 60
    for signal, value in signals.items():
        text = f"{signal}: {value:.2f}"
        cv2.putText(frame, text, (10, y_pos),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (200, 200, 200), 1)
        y_pos += 20

    # Alert
    if alert_level in ["MEDIUM", "HIGH"]:
        timestamp = datetime.now().strftime("%H:%M:%S")
        print(f"[{timestamp}] {alert_level} ALERT | Score: {total_score:.2f} | "
              f"Signals: {signals}")
        evidence = f"evidence/behavior_{datetime.now().strftime('%H%M%S')}.jpg"
        cv2.imwrite(evidence, frame)

    cv2.imshow("ExamGuard v0.3 - Behavior Monitor", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

Success Criteria

Metric	Target
Multi-signal detection	All 4 signals working
Confidence scoring	Correct level for test scenarios
False alarm rate	< 8% (lower than v0.1 and v0.2)
Detection of combined behaviors	> 85%
Processing speed	> 15 fps on GPU

What Comes Next

v0.3 works on ONE camera with ONE student (or a few). The next version (v1.0) adds:

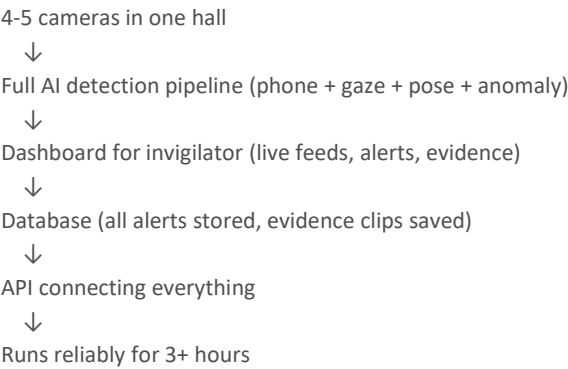
- Multiple cameras (4-5)
- A real dashboard for the invigilator
- Database storage
- API for communication
- This is the jump from prototype to PRODUCT.

Version 1.0 — Single Hall System

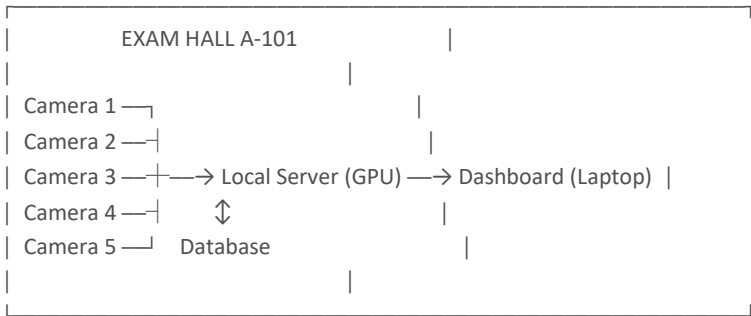
BUILD THIS AFTER: Phase 7 (System Building)

What This Version Does

This is the REAL product. Not a demo. Not a prototype. A system you can actually install in ONE exam hall and use during a real exam.



System Architecture



Local Server: NVIDIA GPU desktop or workstation

- Runs all AI models
- Hosts the API
- Runs the database
- Stores evidence clips

Dashboard Laptop: The invigilator's screen

- Opens browser to <http://server-ip:3000>
- Shows live camera grid
- Shows alerts with evidence
- Confirm/dismiss buttons

Hardware Requirements

Component	Specification	Estimated Cost
GPU Server	NVIDIA RTX 3080+ (10GB+) 16GB RAM, 1TB SSD	\$1,500-2,500
IP Cameras (x5)	1080p, RTSP support	\$100-200 each

Network Switch	Gigabit, PoE (powers cams)	\$100-200
Invigilator Laptop	Any modern laptop	Already have
Ethernet Cables	Cat6, various lengths	\$30-50
UPS (Battery Backup)	30 minutes backup	\$200-400

TOTAL ESTIMATED: \$2,500-4,000

What to Build: Component by Component

Component 1: Camera Manager

"""

Manages all camera connections.

Handles disconnects, reconnects, and health monitoring.

"""

import cv2

import threading

import time

from collections import deque

class CameraManager:

def __init__(self):

self.cameras = {}

self.frames = {}

self.status = {}

def add_camera(self, cam_id, rtsp_url, name):

"""Add a camera to the system."""

self.cameras[cam_id] = {

 'url': rtsp_url,

 'name': name,

 'cap': None,

}

self.frames[cam_id] = None

self.status[cam_id] = 'connecting'

Start reading in background

thread = threading.Thread(

 target=self._read_loop, args=(cam_id,), daemon=True

)

thread.start()

def _read_loop(self, cam_id):

"""Continuously read frames from camera."""

while True:

 try:

 cap = cv2.VideoCapture(self.cameras[cam_id]['url'])

 if not cap.isOpened():

 self.status[cam_id] = 'offline'

 time.sleep(5)

 continue

```

        self.status[cam_id] = 'online'

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        self.frames[cam_id] = frame

    self.status[cam_id] = 'reconnecting'
    cap.release()
    time.sleep(2)

    except Exception as e:
        self.status[cam_id] = 'error'
        time.sleep(5)

    def get_frame(self, cam_id):
        return self.frames.get(cam_id)

    def get_all_status(self):
        return {cid: self.status[cid] for cid in self.cameras}

# Usage:
cam_mgr = CameraManager()
cam_mgr.add_camera(1, "rtsp://192.168.1.10:554/stream1", "Front-Left")
cam_mgr.add_camera(2, "rtsp://192.168.1.11:554/stream1", "Front-Right")
cam_mgr.add_camera(3, "rtsp://192.168.1.12:554/stream1", "Back-Left")
cam_mgr.add_camera(4, "rtsp://192.168.1.13:554/stream1", "Back-Right")
cam_mgr.add_camera(5, "rtsp://192.168.1.14:554/stream1", "Center")

```

Component 2: AI Pipeline

```

"""
Processes frames from all cameras through the full AI pipeline.
"""

```

```

class AIPipeline:
    def __init__(self):
        self.yolo = YOLO("phone_detector.pt")
        self.face_mesh = mp.solutions.face_mesh.FaceMesh(...)
        self.pose = mp.solutions.pose.Pose(...)
        self.autoencoder = load_autoencoder("autoencoder.pt")
        self.scorer = ConfidenceScorer()

    def process_frame(self, frame, camera_id):
        """Run full AI analysis on a single frame."""

        # 1. Object detection
        phone_result = self.detect_phones(frame)

        # 2. Gaze tracking

```

```

gaze_result = self.track_gaze(frame)

# 3. Body pose
pose_result = self.analyze_pose(frame)

# 4. Anomaly check
anomaly_result = self.check_anomaly(frame)

# 5. Combine scores
signals = {
    'phone_detected': phone_result.confidence,
    'gaze_at_neighbor': gaze_result.score,
    'body_leaning': pose_result.lean_score,
    'hands_not_writing': 0.0 if pose_result.is_writing else 1.0,
    'anomaly_score': anomaly_result.score,
}

total_score, alert_level = self.scorer.calculate(signals)

return {
    'camera_id': camera_id,
    'score': total_score,
    'level': alert_level,
    'signals': signals,
    'detections': phone_result.detections,
    'frame': frame
}

```

Component 3: Alert Manager

"""

Manages alerts: creates, stores, sends to dashboard.

"""

```

class AlertManager:
    def __init__(self, db, evidence_dir="/evidence"):
        self.db = db
        self.evidence_dir = evidence_dir
        self.active_alerts = []

    def create_alert(self, ai_result, exam_id):
        """Create a new alert from AI pipeline result."""
        if ai_result['level'] in ['MEDIUM', 'HIGH']:
            # Save evidence clip
            evidence_path = self.save_evidence(ai_result['frame'], ai_result)

            # Create alert record
            alert = {
                'exam_id': exam_id,
                'camera_id': ai_result['camera_id'],
                'alert_type': self.determine_type(ai_result['signals']),
                'confidence': ai_result['score'],
            }

```

```

        'priority': ai_result['level'].lower(),
        'evidence_path': evidence_path,
        'signals': ai_result['signals'],
        'status': 'pending'
    }

    # Save to database
    alert_id = self.db.save_alert(alert)
    alert['id'] = alert_id

    # Add to active list
    self.active_alerts.append(alert)

    return alert

    return None

def determine_type(self, signals):
    """Determine the primary alert type from signals."""
    max_signal = max(signals, key=signals.get)
    type_map = {
        'phone_detected': 'phone_on_desk',
        'gaze_at_neighbor': 'sustained_gaze',
        'body_leaning': 'suspicious_posture',
        'anomaly_score': 'unusual_behavior',
    }
    return type_map.get(max_signal, 'general_suspicion')

```

Component 4: Main Orchestrator

```

"""
ExamGuard v1.0 — Single Hall System
Main entry point that coordinates all components.
"""

import time
import logging

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger('examguard')

class ExamGuardV1:
    def __init__(self, config):
        self.config = config
        self.camera_mgr = CameraManager()
        self.ai_pipeline = AIPipeline()
        self.alert_mgr = AlertManager(database, evidence_dir)
        self.running = False

    # Setup cameras from config
    for cam in config['cameras']:

```

```

        self.camera_mgr.add_camera(cam['id'], cam['url'], cam['name'])

def start(self):
    """Start monitoring."""
    self.running = True
    logger.info("ExamGuard v1.0 starting...")
    logger.info(f"Monitoring {len(self.config['cameras'])} cameras")

    frame_count = 0

    while self.running:
        for cam_id in self.camera_mgr.cameras:
            frame = self.camera_mgr.get_frame(cam_id)
            if frame is None:
                continue

            frame_count += 1

            # Skip frames for performance
            if frame_count % 3 != 0:
                continue

            # Run AI pipeline
            result = self.ai_pipeline.process_frame(frame, cam_id)

            # Create alert if needed
            alert = self.alert_mgr.create_alert(result, self.config['exam_id'])

            if alert:
                logger.info(f"ALERT [{alert['priority'].upper()}]: "
                           f"Camera {cam_id}, {alert['alert_type']}, "
                           f"confidence {alert['confidence']:.2f}")

def stop(self):
    """Stop monitoring and generate report."""
    self.running = False
    report = self.generate_report()
    logger.info("ExamGuard stopped. Report generated.")
    return report

def generate_report(self):
    """Generate end-of-exam report."""
    alerts = self.alert_mgr.active_alerts
    return {
        'total_alerts': len(alerts),
        'high_priority': sum(1 for a in alerts if a['priority'] == 'high'),
        'medium_priority': sum(1 for a in alerts if a['priority'] == 'medium'),
        'confirmed': sum(1 for a in alerts if a['status'] == 'confirmed'),
        'dismissed': sum(1 for a in alerts if a['status'] == 'dismissed'),
        'pending': sum(1 for a in alerts if a['status'] == 'pending'),
    }

```

```
# Run it:
config = {
    'exam_id': 1,
    'exam_name': 'Physics Final',
    'hall': 'A-101',
    'cameras': [
        {'id': 1, 'url': 'rtsp://192.168.1.10:554/stream1', 'name': 'Front-Left'},
        {'id': 2, 'url': 'rtsp://192.168.1.11:554/stream1', 'name': 'Front-Right'},
        {'id': 3, 'url': 'rtsp://192.168.1.12:554/stream1', 'name': 'Back-Left'},
        {'id': 4, 'url': 'rtsp://192.168.1.13:554/stream1', 'name': 'Back-Right'},
    ]
}
```

```
examguard = ExamGuardV1(config)
examguard.start()
```

Deployment Checklist for v1.0

PRE-EXAM (1 day before):

- ☐ Install cameras in hall
- ☐ Verify every seat is visible from at least one camera
- ☐ Test network connectivity
- ☐ Run ExamGuard for 30 minutes — verify stable
- ☐ Test dashboard on invigilator laptop
- ☐ Verify UPS battery is charged
- ☐ Prepare consent forms for students

EXAM DAY (1 hour before):

- ☐ Power on server and cameras
- ☐ Run health check (all cameras online, GPU working, database connected)
- ☐ Open dashboard on invigilator laptop
- ☐ Brief invigilator on dashboard usage
- ☐ Distribute and collect consent forms

DURING EXAM:

- ☐ Monitor system health every 30 minutes
- ☐ Be available for technical issues
- ☐ Invigilator reviews and acts on alerts

AFTER EXAM:

- ☐ Generate exam report
- ☐ Review all pending alerts with invigilator
- ☐ Back up evidence and database
- ☐ Collect feedback from invigilator
- ☐ Document issues for improvement

Success Criteria for v1.0

Metric	Target
Cameras: 4-5 connected	All online

Processing: Real-time	> 10 fps per camera
Uptime: 3-hour exam	> 99% (< 2 min total downtime)
Dashboard: Responsive	Updates < 2 seconds
Alerts: Accurate	> 80% precision
Evidence: Saved	Every alert has video clip
Report: Generated	Automatic end-of-exam report

This IS ExamGuard

v1.0 is not a toy or a demo. It is a deployable product.

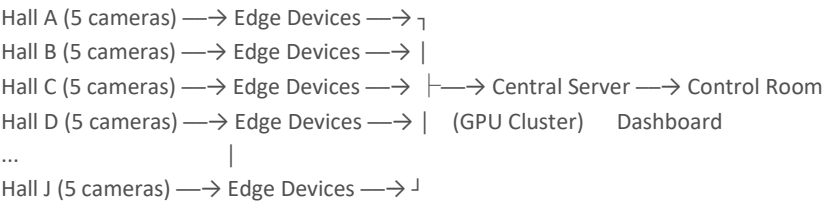
After successful pilot testing with v1.0, you scale to v2.0: multiple halls, edge computing, full production deployment.

Version 2.0 — Multi-Hall System

BUILD THIS AFTER: Phase 8 (Testing)

What This Version Does

The COMPLETE ExamGuard product. Multiple exam halls, dozens of cameras, central control room, edge computing, full reporting, and production-grade reliability.

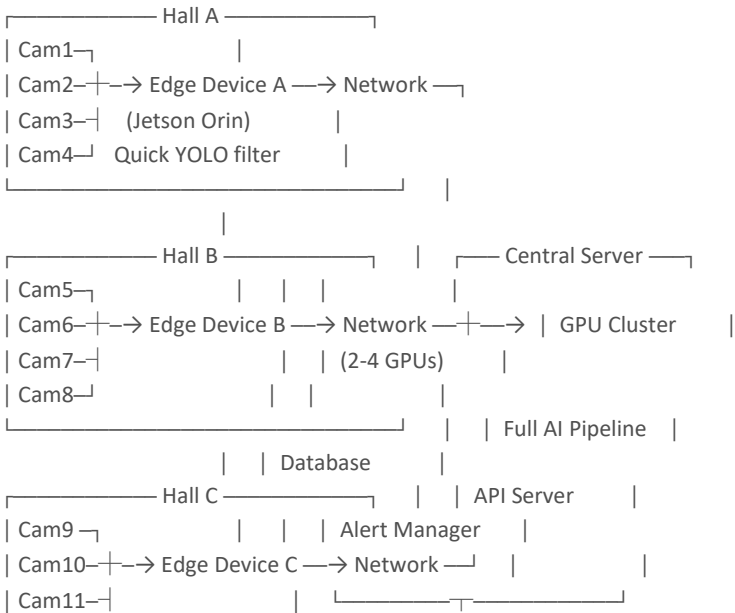


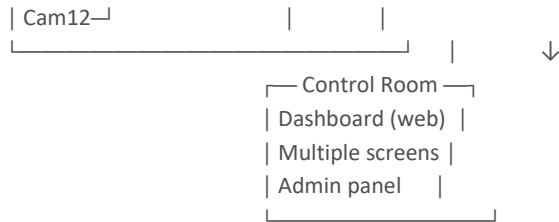
50+ cameras → Edge filtering → Central AI → Unified Dashboard

What Changes from v1.0 to v2.0

Feature	v1.0 (Single Hall)	v2.0 (Multi-Hall)
Cameras	4-5	50-200
Halls	1	10-50
Processing	1 GPU server	GPU cluster + edge devices
Dashboard	1 invigilator screen	Central control room
Database	Local SQLite/PostgreSQL	Distributed PostgreSQL
Network	Simple LAN	Campus-wide network
Redundancy	None	Failover, backups
Management	Manual start	Automated management
Reports	Per-exam	Per-institution, analytics

System Architecture





Key Components to Build

Component 1: Edge Device Software

Each hall gets 1-2 edge devices (NVIDIA Jetson) that pre-filter frames.

"""

Edge device script — runs on Jetson at each hall.

Filters out normal frames, sends only suspicious ones to central server.

"""

```
class EdgeProcessor:
```

```
    def __init__(self, hall_id, camera_urls, server_url):
```

```
        self.hall_id = hall_id
```

```
        self.model = YOLO("yolov8n.engine") # TensorRT optimized
```

```
        self.server_url = server_url
```

```
        self.cameras = {}
```

```
    for cam_id, url in camera_urls.items():
```

```
        self.cameras[cam_id] = cv2.VideoCapture(url)
```

```
    def run(self):
```

```
        """Main processing loop."""
```

```
        frame_count = 0
```

```
        while True:
```

```
            for cam_id, cap in self.cameras.items():
```

```
                ret, frame = cap.read()
```

```
                if not ret:
```

```
                    continue
```

```
            frame_count += 1
```

```
            if frame_count % 3 != 0: # Process every 3rd frame
```

```
                continue
```

```
            # Quick YOLO check
```

```
            results = self.model(frame, verbose=False)
```

```
            is_suspicious = self.check_suspicious(results)
```

```
            if is_suspicious:
```

```
                # Send to central server for deep analysis
```

```
                self.send_to_server(frame, cam_id, results)
```

```
            # Always send heartbeat every 30 seconds
```

```

        if frame_count % 900 == 0: # 30fps * 30sec
            self.send_heartbeat(cam_id)

def check_suspicious(self, results):
    """Quick check: anything worth sending to server?"""
    suspicious_objects = ['cell phone', 'book']
    for box in results[0].boxes:
        class_name = self.model.names[int(box.cls)]
        if class_name in suspicious_objects and float(box.conf) > 0.5:
            return True
    return False

def send_to_server(self, frame, cam_id, results):
    """Send suspicious frame to central server."""
    _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 85])
    try:
        requests.post(
            f"{self.server_url}/api/edge/frame",
            files={'frame': buffer.tobytes()},
            data={
                'hall_id': self.hall_id,
                'camera_id': cam_id,
                'timestamp': time.time(),
            },
            timeout=3
        )
    except requests.exceptions.RequestException:
        pass # Log error, continue processing

```

Component 2: Central Server API

```

"""
Central server receives frames from edge devices and runs full analysis.
"""

```

```

from fastapi import FastAPI, UploadFile, File, Form

app = FastAPI(title="ExamGuard Central Server v2.0")

# Load full AI models (more powerful than edge)
full_pipeline = FullAIPipeline()

@app.post("/api/edge/frame")
async def receive_edge_frame(
    frame: UploadFile = File(...),
    hall_id: str = Form(...),
    camera_id: int = Form(...),
    timestamp: float = Form(...)
):
    """Receive a suspicious frame from an edge device."""
    # Decode frame
    contents = await frame.read()

```

```

nparr = np.frombuffer(contents, np.uint8)
image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

# Run full AI pipeline
result = full_pipeline.process(image, hall_id, camera_id)

if result['level'] in ['MEDIUM', 'HIGH']:
    # Create alert
    alert_manager.create_alert(result)

return {"status": "processed", "result": result['level']}

@app.get("/api/halls")
def list_halls():
    """List all halls and their status."""
    return hall_manager.get_all_status()

@app.get("/api/halls/{hall_id}/alerts")
def get_hall_alerts(hall_id: str):
    """Get alerts for a specific hall."""
    return alert_manager.get_alerts_by_hall(hall_id)

@app.get("/api/dashboard/summary")
def dashboard_summary():
    """Get summary for the central control room dashboard."""
    return {
        'halls_online': hall_manager.count_online(),
        'halls_total': hall_manager.count_total(),
        'cameras_online': camera_manager.count_online(),
        'cameras_total': camera_manager.count_total(),
        'active_alerts': alert_manager.count_active(),
        'alerts_today': alert_manager.count_today(),
        'confirmed_today': alert_manager.count_confirmed_today(),
        'system_health': health_monitor.get_status()
    }

```

Component 3: Multi-Hall Dashboard

The control room dashboard shows:

ExamGuard v2.0 — Central Control Room									
Halls: 10/10 online		Cameras: 48/50 online			Alerts: 12				
HALL OVERVIEW					ACTIVE ALERTS				
Hall A: 5/5 cams		2	Hall C Cam 3 - Phone						
Hall B: 5/5 cams		0	Seat B4 92% 14:23:05						
Hall C: 4/5 cams		3	[View] [Assign] [Dismiss]						
Hall D: 5/5 cams		1							
Hall E: 5/5 cams		0	Hall A Cam 2 - Gaze						

Hall F:	3/5 cams	0	Seat D1	78%	14:22:50	
Hall G:	5/5 cams	2	[View]	[Assign]	[Dismiss]	
Hall H:	5/5 cams	1				
Hall I:	5/5 cams	0	Hall G Cam 1 - Anomaly			
Hall J:	5/5 cams	3	Seat A3	65%	14:22:30	
			[View]	[Assign]	[Dismiss]	
Click hall to see cameras						
			+ 9 more alerts...			
TODAY'S STATS: 87 alerts 62 confirmed 18 dismissed 7 						

Component 4: Analytics and Reporting

Post-exam analytics for v2.0.

```
class AnalyticsEngine:
    def generate_institution_report(self, date):
        """Generate report across ALL halls for a given date."""
        return {
            'date': date,
            'halls_monitored': 10,
            'total_students': 2000,
            'total_alerts': 87,
            'confirmed_cheating': 62,
            'false_alarms': 18,
            'pending': 7,
            'precision': 62 / (62 + 18), # 77.5%
            'detection_types': {
                'phone_detected': 25,
                'sustained_gaze': 30,
                'note_passing': 5,
                'unusual_behavior': 15,
                'other': 12
            },
            'hall_comparison': {
                'Hall A': {'alerts': 12, 'confirmed': 9},
                'Hall B': {'alerts': 5, 'confirmed': 4},
                # ...
            },
            'camera_performance': {
                'most_alerts': 'Hall C Cam 3 (15 alerts)',
                'cameras_offline_during_exam': ['Hall F Cam 4', 'Hall F Cam 5'],
            },
            'system_performance': {
                'average_alert_latency': '1.8 seconds',
                'system_uptime': '99.7%',
                'total_frames_processed': 5400000,
            },
            'recommendations': [
```

```

    'Hall F network needs repair (2 cameras offline)',
    'Camera 3 in Hall C may need repositioning (too many alerts)',
    'Consider adding camera to Hall B corner (blind spot reported)',
  ]
}

```

Scaling Considerations

Network Architecture

Campus Network:

Core Switch (10 Gbps)

↓

Building Switches (1 Gbps)

↓

Hall Switches (1 Gbps, PoE)

↓

Cameras (100 Mbps each)

Bandwidth calculation:

50 cameras × 5 Mbps each (compressed) = 250 Mbps

With edge filtering (only 10% sent): 25 Mbps to central server

Well within 1 Gbps capacity

Server Scaling

Small deployment (10-20 cameras):

1 server, 1 GPU (RTX 3080)

Medium deployment (20-50 cameras):

1 server, 2 GPUs (RTX 4080)

Edge devices at each hall

Large deployment (50-200 cameras):

2 servers, 4 GPUs total

Edge devices at each hall

Load balancer distributes work

Enterprise deployment (200+ cameras):

Cloud-based GPU cluster

Edge devices required at every hall

Distributed database

Multiple dashboard instances

Reliability

v1.0: If server crashes → monitoring stops

v2.0: Redundancy at every level

Camera fails → Other cameras in the hall cover

Edge device fails → Frames go directly to central server (slower but works)

Central GPU fails → Second GPU takes over

Database fails → Redis cache maintains recent alerts

Network fails → Edge devices store locally, sync when network returns

Dashboard fails → Secondary dashboard URL available
Power fails → UPS at each hall gives 30 minutes

Development Timeline (Realistic)

After completing v1.0 and successful pilot testing:

Month 1: Design v2.0 architecture, order hardware
Month 2: Build edge device software, test on Jetson
Month 3: Build central server, multi-hall API
Month 4: Build control room dashboard
Month 5: Build analytics and reporting
Month 6: Integration testing with 2-3 halls
Month 7: Pilot testing with real exams
Month 8: Fix issues from pilot, optimize
Month 9: Scale to 5-10 halls
Month 10: Full deployment

Success Criteria for v2.0

Metric	Target
Halls supported	10+ simultaneously
Cameras per hall	5
Total cameras	50+
Processing latency	< 3 seconds per alert
System uptime	> 99.9% (< 3 min downtime per exam)
Alert precision	> 85%
Alert recall	> 90%
Edge filtering rate	> 85% of frames filtered
Central server GPU utilization	< 80%
Dashboard response time	< 1 second
Report generation	Automatic within 5 minutes of exam end

This Is the Complete Product

v2.0 is what you pitch to universities and exam boards. It is:

- Scalable: Works for 10 students or 10,000
- Reliable: Redundancy at every level
- Ethical: Human-in-the-loop, consent, data protection
- Useful: Real-time alerts, evidence clips, analytics

From learning "what is a neural network" to deploying a multi-hall AI surveillance system — this is the complete journey.

MVP Versions — Learning Resources

Similar Products to Study

Study these existing products to understand what the market expects:

Commercial AI Proctoring Systems

Proctorio

- What it does: Browser-based exam proctoring, webcam monitoring, screen recording
- How it works: AI flags suspicious behavior, human reviews
- Study: Their website, YouTube demos, user reviews (learn from complaints too!)
- Website: proctorio.com

ExamSoft (Examplify)

- What it does: Secure exam browser + AI monitoring
- How it works: Locks down computer + monitors via webcam
- Study: How they handle false positives, their reporting features
- Website: examsoft.com

Respondus LockDown Browser + Monitor

- What it does: Locks browser + webcam recording + AI flagging
- How it works: Records video, AI scans for suspicious movements post-exam
- Study: Their AI flagging system, how invigilators review flags
- Website: respondus.com

Mercer Mettl

- What it does: Online exam proctoring with AI + human proctors
- How it works: Three levels — AI only, AI + human review, live human proctor
- Study: Their tiered approach is smart — different exams need different levels
- Website: mettl.com

ProctorU / Meazure Learning

- What it does: Live human proctoring augmented with AI
- How it works: Human proctor watches via webcam, AI assists with detection
- Study: How they combine human and AI monitoring
- Website: meazurelearning.com

What to Learn From Them

1. User interface — How do they show alerts? What does the dashboard look like?
2. Privacy approach — How do they handle consent and data?
3. Accuracy claims — What accuracy do they claim? How do they measure?
4. Complaints — Search Reddit and forums for complaints (learn from their mistakes)
5. Pricing — How do they charge? Per student? Per exam? Per institution?
6. Edge cases — How do they handle disabilities, technical issues?

Open Source Projects to Reference

Object Detection

- YOLOv8 (Ultralytics): github.com/ultralytics/ultralytics — The model you will use
- YOLO-NAS: github.com/Deci-AI/super-gradients — Alternative to YOLO
- Detectron2: github.com/facebookresearch/detectron2 — Facebook's detection framework

Pose Estimation

- MediaPipe: google.github.io/mediapipe — Face mesh, pose, hands (Google)
- OpenPose: github.com/CMU-Perceptual-Computing-Lab/openpose — Body pose estimation
- AlphaPose: github.com/MVIG-SJTU/AlphaPose — Multi-person pose estimation

Anomaly Detection

- PyOD: github.com/yzhao062/pyod — Python Outlier Detection library (30+ algorithms)
- Alibi Detect: github.com/SeldonIO/alibi-detect — Outlier and drift detection

Dashboard / Web

- Streamlit: streamlit.io — Build data dashboards in Python (fastest way to prototype)
- Grafana: grafana.com — Professional monitoring dashboards (for system metrics)
- Flask: flask.palletsprojects.com — Lightweight web framework

Deployment

- Docker: docker.com — Containerization
- NVIDIA Triton: github.com/triton-inference-server — Model serving at scale
- BentoML: github.com/bentoml/BentoML — ML model deployment

YouTube Videos

How AI Proctoring Works

- "How Online Exam Proctoring Works" by Vice — Shows the student experience
- "AI Proctoring Software Explained" by WIRED — Technical overview
- "The Problem with Online Exam Proctoring" by Tom Scott — Ethical concerns (important to understand!)

Building Each Version

- "Object Detection with YOLOv8 — Complete Tutorial" — For v0.1
- "Eye Tracking with Python and MediaPipe" — For v0.2
- "Building a Real-Time Dashboard with Flask" — For v1.0
- "Deploying ML Models with Docker" — For v1.0/v2.0
- "NVIDIA Jetson for Edge AI" — For v2.0

System Design

- "How to Design a Real-Time System" by Gaurav Sen — Architecture thinking
- "System Design for ML" by Chip Huyen — ML-specific system design

- "Microservices Explained" by TechWorld with Nana — Why separate components

Version-Specific Resources

v0.1 Phone Detector

Must learn: YOLOv8 fine-tuning, data labeling

YouTube: "Custom Object Detection YOLOv8" by Nicolai Nielsen

Tool: Roboflow (roboflow.com) for data labeling

Dataset: Search Kaggle for "phone detection" datasets

v0.2 Gaze Tracker

Must learn: MediaPipe Face Mesh, eye landmark detection

YouTube: "Gaze Detection Python MediaPipe" by Murtaza's Workshop

Documentation: google.github.io/mediapipe/solutions/face_mesh

Paper: "MPIIGaze: Real-World Dataset for Gaze Estimation"

v0.3 Behavior Monitor

Must learn: MediaPipe Pose, signal combination, confidence scoring

YouTube: "Pose Estimation MediaPipe Python" by Nicholas Renotte

YouTube: "Autoencoder for Anomaly Detection" by Krish Naik

Practice: Combine multiple MediaPipe solutions in one pipeline

v1.0 Single Hall

Must learn: Flask/FastAPI, database, multi-camera, Docker

YouTube: "Full Stack ML Project" by Patrick Loeber

YouTube: "FastAPI + React Dashboard" by various

Practice: Build each component standalone, then integrate

v2.0 Multi-Hall

Must learn: Edge computing, distributed systems, scaling

YouTube: "NVIDIA Jetson AI Projects" by JetsonHacks

YouTube: "System Design Interview" by Gaurav Sen

Practice: Deploy v1.0 first, then plan v2.0 architecture

Project Milestones Checklist

Use this to track your progress:

- ☐ Phase 1-4: Foundations (Python, Math, ML basics)
- ☐ Phase 5: Object Detection & Classification
 - ☐ v0.1: Phone Detector BUILT AND WORKING
 - ☐ v0.2: Gaze Tracker BUILT AND WORKING
- ☐ Phase 6: Advanced ML (LSTM, Autoencoder, RL)
 - ☐ v0.3: Behavior Monitor BUILT AND WORKING
- ☐ Phase 7: System Building (Camera, Dashboard, API, Docker)
 - ☐ v1.0: Single Hall System BUILT AND WORKING
- ☐ Phase 8: Testing (Edge cases, Ethics, Performance, Pilot)
 - ☐ v1.0: Pilot tested in real exam
 - ☐ v2.0: Multi-Hall System DESIGNED
 - ☐ v2.0: Multi-Hall System BUILT AND TESTED
 - ☐ v2.0: Deployed in production

Final Advice

1. BUILD v0.1 as soon as possible

Do not wait until you "know everything." Build, fail, learn, improve.

2. Show people your work at EVERY version

Feedback from real users is worth more than 100 hours of solo coding.

3. Each version should be USABLE

v0.1 is useful. v0.2 is useful. Do not wait for v2.0 to be "complete."

4. Study the competitors

They have spent millions learning what works. Learn from their experience.

5. Ethics are not a phase — they are ongoing

Think about privacy and fairness at EVERY version, not just Phase 8.

6. The journey from v0.1 to v2.0 takes 12-18 months

This is normal. Professional products take years. Be patient.